

NixOS Swarm Cluster

Setup Guide — self-bootstrapping nodes with 1Password, Tailscale, Ceph, Docker Swarm & Portainer

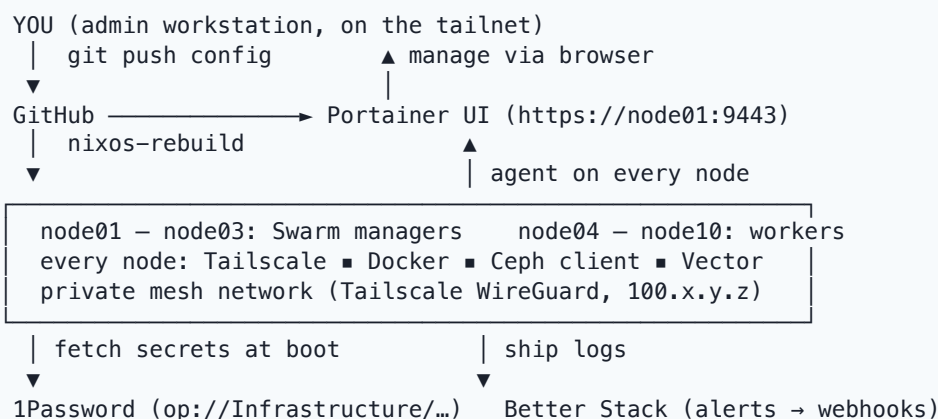
Version 1.0 · June 2026 · Companion to github.com/SEBK4C/NixOS · Written for junior DevOps engineers — no NixOS experience assumed

Contents

1. What you're building
2. Accounts & URLs you need
3. Step 1 — Set up 1Password (vaults, items, service account)
4. Step 2 — Set up Tailscale
5. Step 3 — Set up the Git repositories
6. Step 4 — Bring up the first node
7. Step 5 — Store Swarm tokens, bring up remaining nodes
8. Step 6 — Deploy Portainer
9. Step 7 — Ship logs to Better Stack
10. Step 8 — Validate every node
11. Day-2 operations
12. Scaling to Dev / Staging / Production
13. Troubleshooting
14. Appendix A — Complete credential inventory

1. What you're building

A private cluster of ten NixOS machines that **configure themselves**. You point a fresh machine at a Git repository, give it one secret (a 1Password token), and it does the rest: fetches its credentials, joins your private network, mounts shared storage, joins the container cluster, and reports in to a management UI.



The three rules that make this safe

1. **Git is the source of truth, and it holds zero secrets.** The public repo is the framework anyone can reuse; your private *ops* repo holds your real inventory. Neither ever contains a password, key, or token.
2. **One secret per machine.** The only credential you ever place on a node by hand is a 1Password *service account token*. Everything else (Tailscale keys, Ceph keys, Swarm tokens, logging tokens) is

fetches from 1Password at boot, by reference.

3. **Nothing listens on the public internet.** All management traffic flows over the Tailscale mesh. The firewall trusts only the tailnet interface.

2. Accounts & URLs you need

Service	Used for	Sign up / console URL	Cost
1Password	All cluster secrets, fetched at runtime	1password.com — service accounts at my.1password.com → Developer	Personal/Family plan works; service accounts included
Tailscale	Private WireGuard mesh between nodes	login.tailscale.com — keys at /admin/settings/keys	Free tier covers up to 100 devices
GitHub	Public framework repo + private ops repo	github.com — PATs at github.com/settings/personal-access-tokens/new	Free (private repos included)
Better Stack	Log aggregation, alerting, webhooks	betterstack.com — sources at telemetry.betterstack.com	Free tier: 3 GB logs/month
Anthropic or OpenAI	API key for the (planned) support agent	console.anthropic.com/settings/keys · platform.openai.com/api-keys	Pay per use; optional until the agent ships
Ceph cluster	Shared persistent storage (optional at start)	docs.ceph.com — yours to host; the cluster only needs client access	Self-hosted

Hardware: 10 x86_64 machines (VMs or bare metal) with NixOS minimal installed — installer at **nixos.org/download**. The node count is configurable in the flake; three managers is the minimum for a fault-tolerant Swarm.

3. Step 1 — Set up 1Password

3.1 Create the vaults

Create **two vaults** (1Password app → New Vault):

Vault	Purpose	Who can read it
Infrastructure	Runtime secrets the nodes fetch at boot. Item names below are read by the code <i>exactly as written</i> — no spaces, match the capitalization.	You + the cluster's service account (read-only)
Admin	Master copies and break-glass credentials: the service-account token itself, account logins, recovery codes. Never grant any service account access to this vault.	You only

Why two vaults?

If a node is ever compromised, the attacker gets the node's service-account token — which can read *only* the Infrastructure vault. Your account logins, recovery codes, and the token master copies stay out of reach, and you revoke one service account to lock everything down.

3.2 Create the Infrastructure vault items

Create each item as type *API Credential* (or *Secure Note* for Environment). Field names matter — the nodes read `op://Infrastructure/<Item>/<field>` literally.

Item name	Field	What goes in it	Where you get it	When
Tailscale	credential	Auth key (tskey-auth-...) — set <i>Reusable</i> + <i>Pre-approved</i> , 90-day expiry	login.tailscale.com/admin/settings/keys → Generate auth key	Before first node
Ceph	credential	CephFS client secret key (base64 string only, not the full keyring)	On your Ceph cluster: ceph auth get- key client.swarm	When enabling storage (optional)
Swarm	manager- token	SWMTKN-... manager join token	node01's journal after first boot (Step 5)	After node01 is up
Swarm	worker- token	SWMTKN-... worker join token	Same place	After node01 is up
BetterStack	credential	Telemetry source token	telemetry.betterstack.com → Sources → Connect source (platform: Vector/HTTP)	When enabling logging
GitHub	credential	Fine-grained PAT, read-only : <i>Contents: Read</i> on your private ops repo only	github.com/settings/personal-access- tokens/new	Before first node
SupportAgent	github- token	Fine-grained PAT, read-write: <i>Contents + Issues</i> + <i>Pull requests</i> on the ops repo; <i>Contents + Pull</i> <i>requests</i> on the framework repo	Same URL	When the agent ships
SupportAgent	llm-api-key	Anthropic or OpenAI API key	console.anthropic.com/settings/keys or platform.openai.com/api-keys	When the agent ships
SupportAgent	webhook- secret	Random string you generate: openssl rand -hex 32	Your terminal	When the agent ships
SupportAgent	betterstack- api-token	Better Stack API token (for the agent's log queries)	Better Stack → settings → API tokens	When the agent ships

Environment	notes	Non-secret, host-specific config overrides (free text)	You write it	Optional
--------------------	-------	--	--------------	----------

3.3 Create the service account

1. Go to **my.1password.com** → **Developer** → **Service Accounts** → **Create** (docs: developer.1password.com/docs/service-accounts).
2. Name it after the environment, e.g. `nixos-cluster-prod`.
3. Grant **Read** access to the **Infrastructure** vault. Nothing else. Not the Admin vault.
4. The token (`ops_...`) is shown **once**. Immediately save it in the **Admin** vault as item **Service Account Token — Production**.

3.4 Create the Admin vault items

Item name	Contents
Service Account Token — Production	The <code>ops_...</code> token from 3.3 (master copy; you'll paste it onto each node once)
Tailscale Admin	Your Tailscale login method, tailnet name
GitHub Account	Login, 2FA recovery codes (enable passkeys/2FA!)
Better Stack Login	Login credentials
Portainer Admin	The admin password you'll create at first Portainer login (Step 6)
LLM Provider	Anthropic/OpenAI account login (the API key's master home)

4. Step 2 — Set up Tailscale

1. Create a tailnet at **login.tailscale.com** and install Tailscale on your admin workstation (your laptop must be on the tailnet to manage the cluster).
2. Check **DNS** → **MagicDNS** is enabled (it is by default) — nodes find each other by hostname (`node01`) instead of IP.
3. Generate the auth key (**Settings** → **Keys** → **Generate auth key**): *Reusable* ✓, *Pre-approved* ✓ (so nodes don't wait for manual device approval), expiry 90 days.
4. Store it in 1Password: **Infrastructure** → **Tailscale** → **credential**.

Key expiry tip.

The auth key only matters when a node *joins*. Existing nodes stay connected after it expires — you only need a fresh key (same 1Password item, updated) when adding or rebuilding nodes.

5. Step 3 — Set up the Git repositories

Two repositories, deliberately split:

Repo	Visibility	Contains
NixOS (the framework — fork <code>github.com/SEBK4C/NixOS</code>)	Public	Shared modules, specs, this guide. Generic and secret-free; safe to publish, easy to share.
NixOS-Ops (your inventory — copy the layout from the framework's README)	Private	Your real settings, per-node hardware configs, host-specific software, workload stacks, and the support-agent issue trail. Private because <i>operational</i> data (logs, addresses, incident details) shouldn't be public, even though it also contains no secrets.

1. Fork the framework repo (or use it directly if you're me).
2. Create your private ops repo with: `flake.nix` (composes the framework's `nixosModules.cluster`), `cluster-settings.nix`, and a `hosts/nodeXX/` directory per node — see `github.com/SEBK4C/NixOS-Ops` layout in the framework README.
3. Edit `cluster-settings.nix` in the ops repo: your SSH public key in `adminSSHKeys`, Ceph monitor addresses (or leave empty to skip Ceph), `opsFlake` pointing at your ops repo.
4. Create the **read-only** GitHub PAT (Section 3.2, item **GitHub**) — nodes use it to fetch the private repo. Use a fine-grained token scoped to *only* the ops repo with *Contents: Read*.
5. On GitHub, enable **passkeys/2FA** and **branch protection** on `main` of both repos. Whoever controls these repos controls your machines.

6. Step 4 — Bring up the first node

1. **Install NixOS minimal** on the machine (nixos.org/download → minimal ISO). Any standard install works; give it disk and network.
2. **Capture the hardware config** — on the node:

```
nixos-generate-config --show-hardware-config > hardware-configuration.nix
```

Copy this file into your **ops repo** at `hosts/node01/hardware-configuration.nix`, commit, push. (This is the only per-node file; repeat for each node.)

3. **Place the one manual secret** — on the node, as root:

```
mkdir -p /etc/op
echo 'ops_YOUR_SERVICE_ACCOUNT_TOKEN' > /etc/op/token
chmod 600 /etc/op/token
```

4. **Build** — the first build needs the GitHub token inline (1Password isn't wired up yet):

```
export NIX_CONFIG="experimental-features = nix-command flakes
access-tokens = github.com=github_pat_YOUR_READONLY_TOKEN"
nixos-rebuild switch --flake github:YOUR_USER/NixOS-Ops#node01
```

5. Watch it bootstrap: validates 1Password → joins the tailnet → (mounts Ceph) → initializes the Swarm. Check progress with `journalctl -u op-bootstrap -u tailscale-join -u swarm-join -f`.

From now on, rebuilds are one command.

Every node gets a `cluster-rebuild` helper that fetches the GitHub token from 1Password per run — nothing stored on disk. `cluster-rebuild (apply)`, `cluster-rebuild test` (try without persisting), `cluster-rebuild boot` (apply at next reboot).

7. Step 5 — Swarm tokens, remaining nodes

1. node01 initialized the Swarm on first boot and printed join tokens. Read them:

```
journalctl -u swarm-join | grep -A2 'join-token'
```

2. Store both `SWMTKN-...` values in 1Password: **Infrastructure** → **Swarm** → **manager-token / worker-token**.
3. Repeat Step 4 (install → hardware config → token file → build) for node02...node10, using the matching name (`#node02` etc.). Each node reads the right token for its role and joins by itself. Verify on any manager: `docker node ls` — all nodes *Ready*, 01-03 as managers.

8. Step 6 — Deploy Portainer

1. On node01: `cluster-deploy-portainer` (deploys the Portainer server + an agent on every node, as Swarm services).
2. From your workstation (on the tailnet), open **`https://node01:9443`**, create the admin account, and save the password in the **Admin** vault (item **Portainer Admin**).
3. You should see all ten nodes with CPU/RAM/containers. Portainer is your window, not your source of truth — durable changes still go through Git.

9. Step 7 — Ship logs to Better Stack

1. In **telemetry.betterstack.com** → **Sources** → **Connect source**, create a source (platform: *Vector* or *HTTP*). Copy its **token** and **ingest URL**.
2. Token → 1Password: **Infrastructure** → **BetterStack** → **credential**.
3. Ingest URL → ops repo `cluster-settings.nix` → `logging.endpoint`. Commit, push, then `cluster-rebuild` on each node.
4. Every node now ships its full journal — including all container logs — via Vector. Set up alerts in Better Stack (e.g. on `FAILED` patterns or log-volume anomalies) and point them at e-mail now, webhooks later (that's the support agent's trigger).

Why Better Stack, and the exit ramp.

Chosen for ease of setup: hosted, OpenTelemetry-native, free tier, built-in alerting/webhooks, and an MCP server AI agents can query. Because shipping goes through Vector (vendor-neutral), migrating later to self-hosted Loki or ClickStack is: stand up the backend → change `logging.endpoint` → swap the 1Password credential. Dashboards and alert rules are what you'd recreate.

10. Step 8 — Validate every node

Run on each node before trusting it with real workloads:

Check	Command	Pass looks like
1Password	<code>op-cluster vault list</code>	Lists the Infrastructure vault
Tailscale	<code>tailscale status</code>	Node listed with 100.x address; peers visible
Ceph (if enabled)	<code>mountpoint /mnt/cephfs</code> then write/read a test file	"is a mountpoint"; file round-trips
Swarm	<code>docker node ls</code> (managers)	All nodes Ready, correct roles
Portainer	Browser → node detail	Resources and tasks visible
Logging	Better Stack live tail	This node's journal lines arriving
Workload	Deploy a test service via Portainer with a volume on Ceph	Schedules, starts, persists data

11. Day-2 operations

- **Change anything** → edit the ops repo, push, `cluster-rebuild` on affected nodes. Manual hotfixes on a node must be backported to the repo — an unmerged fix is debt.
- **Per-node software** (GPU drivers, an accelerator card, a one-off service) → create `hosts/nodeXX/extra.nix` in the ops repo. It's auto-imported; shared modules stay untouched.
- **Roll back a bad change** → `nixos-rebuild switch --rollback` on the node returns to the previous generation; then fix the repo.
- **Decommission a node** → drain in Portainer, `docker node rm`, remove from Tailscale admin, **revoke its 1Password service-account token if compromised**.
- **Support agent (planned)** → each node will run a static-binary AI agent (spec: github.com/SEBK4C/Nix-Support-Agent). You'll open an issue in the ops repo labeled `node/node07` ("install ROCm"), and the node's agent diagnoses, implements via PR, and reports back. Better Stack alert webhooks trigger it automatically for incidents.
- **Keep current** → periodically `nix flake update` in the ops repo (updates NixOS packages), test with `cluster-rebuild test` on one worker before rolling out.

12. Scaling to Dev / Staging / Production

The whole design is parameterized by **vault name**. To run multiple environments:

1. Create one vault per environment: `Infra-Dev` , `Infra-Staging` , `Infra-Prod` — each containing the **same item names** from Section 3.2 (different credential values: separate tailnets or ACL-segmented tags, separate Better Stack sources, etc.).
2. Create one service account per environment, each granted read access to **exactly its own vault** (`nixos-cluster-dev` → `Infra-Dev` only). Store each token in the Admin vault.
3. In each environment's ops repo (or branch), set `cluster.vault = "Infra-Dev";` in `cluster-settings.nix` . That single line selects the environment — every `op://` reference follows it.
4. Blast-radius rule: a compromised dev node can never read staging or production secrets, because its token simply has no grant there.

13. Troubleshooting

Symptom	Where to look	Usual cause
1Password auth fails	<code>journalctl -u op-bootstrap</code>	Token expired/typo'd in <code>/etc/op/token</code> ; service account lacks vault grant; system clock skew (check <code>chronyc tracking</code>)
Secret not found	Same	Item or field name doesn't match Section 3.2 exactly (spaces, capitalization)
Tailnet join fails	<code>journalctl -u tailscale-join</code>	Auth key expired or not <i>pre-approved</i> ; ACL policy blocks the node
Ceph mount fails	<code>journalctl -u mnt-cephfs.mount</code>	MON addresses wrong in settings; key in 1Password isn't the bare base64 secret
Swarm join fails	<code>journalctl -u swarm-join</code>	Tokens not yet stored in 1Password (Step 5); manager unreachable over tailnet
Node missing in Portainer	<code>docker service ps portainer_agent</code>	Agent task not scheduled on that node; check node is Ready
No logs in Better Stack	<code>journalctl -u vector -u logging-secret</code>	Endpoint not set; token missing/wrong in 1Password
Private repo fetch fails	Error from <code>nixos-rebuild</code>	GitHub PAT expired or lacks <i>Contents: Read</i> on the ops repo

14. Appendix A — Complete credential inventory

Everything secret in the entire system, in one table. If it's not listed here, it should not exist.

#	Credential	Lives in	Created at	Scope / notes
1	1Password service-account token (ops_...)	Admin vault (master) + /etc/op/token on each node	my.1password.com → Developer → Service Accounts	Read-only on Infrastructure vault. One per environment. The only secret ever placed by hand.
2	Tailscale auth key (tskey-auth-...)	Infrastructure → Tailscale → credential	login.tailscale.com/admin/settings/keys	Reusable, pre-approved, 90-day expiry. Only needed when nodes join.
3	CephFS client key	Infrastructure → Ceph → credential	ceph auth get-key client.swarm on the Ceph cluster	Bare base64 secret. Client mount access only.
4	Swarm manager join token	Infrastructure → Swarm → manager-token	node01 journal after first boot	Joins managers (nodes 01–03)
5	Swarm worker join token	Infrastructure → Swarm → worker-token	Same	Joins workers (nodes 04–10)
6	Better Stack source token	Infrastructure → BetterStack → credential	telemetry.betterstack.com → Sources	Log ingestion only
7	GitHub read-only PAT	Infrastructure → GitHub → credential	github.com/settings/personal-access-tokens/new	Fine-grained; Contents: Read on the private ops repo only. Used by cluster-rebuild .
8	Agent GitHub PAT (read-write)	Infrastructure → SupportAgent → github-token	Same URL	Ops repo: contents/issues/PRs. Framework repo: contents/PRs. No admin scopes.
9	LLM API key	Infrastructure → SupportAgent → llm-api-key	console.anthropic.com/settings/keys or platform.openai.com/api-keys	Support agent's model access
10	Webhook shared secret	Infrastructure → SupportAgent → webhook-secret	openssl rand -hex 32	Authenticates Better Stack → agent webhooks
11	Better Stack API token	Infrastructure → SupportAgent → betterstack-api-token	Better Stack settings → API tokens	Agent's log-query access (MCP)

12	Portainer admin password	Admin vault → Portainer Admin	You, at first login (https://node01:9443)	Human use only; never fetched by nodes
13	Account logins + recovery codes (GitHub, Tailscale, Better Stack, 1Password, LLM provider)	Admin vault	Each provider	Break-glass. Enable 2FA/passkeys everywhere.

Generated for the NixOS Swarm Cluster project · framework: github.com/SEBK4C/NixOS · cluster spec: [prompt.md](#) · agent spec: [github.com/SEBK4C/Nix-Support-Agent](#)